

# Polymorphism

**Concepts of Programming Languages**  
**Lecture 4**

# Practice Problem

let  $x = 2$  in  
 $x + x \rightarrow 2 + 2$

```
let k = fun x -> fun y -> x in  
let x = 3 + k k 2 3 4 in  
k x (k x)
```

*Would UTop throw a type error on the above expressions? If not, what does it evaluate to?*

let  $k = \text{fun } x \rightarrow x + x$  in  
 $k \ 2 \rightarrow (\text{fun } x \rightarrow x + x) \ 2 \rightarrow 2 + 2 \rightarrow 4$

# Answer

```
let k = fun x -> fun y -> x in
let x = 3 + (((k k) 2) 3) 4 in
k x (k x)
```

↳ let x = 3 + (fun x → (fun y → x)) (fun x → fun y → x) 2 3 4 in ---

↳ let x = 3 + (fun y → [fun x → fun y → x]) 2 3 4 in ---

let x = 3 + (fun x → (fun y → x)) 3 4 in ---

let x = 3 + (fun y → 3) 4 in ---

let x = 3 + 3 in k x (k x)

# Answer

```
let k = fun x -> fun y -> x in
let x = 3 + k k 2 3 4 in
k x (k x)
```

let  $x = 6$  in  $k$   $x$  ( $k$   $x$ )

$(\text{fun } x \rightarrow (\text{fun } y \rightarrow x)) \ 6 \ ((\text{fun } x \rightarrow \text{fun } y \rightarrow x) \ 6)$

$(\text{fun } y \rightarrow 6) \ ((\text{fun } x \rightarrow (\text{fun } y \rightarrow x)) \ 6)$

$(\text{fun } y \rightarrow 6) \ \underline{(\text{fun } y \rightarrow 6)}$

6

# Answer

```
let k = fun x -> fun y -> x in
let x = 3 + k k 2 3 4 in
k x (k x)
```

let x = 3 + ((k 2) 3) 4 k in ...

→  
let x = 3 + (fun y → 2) 3 4 k in ...

→  
let x = 3 + 2 4 k in

# Outline

- » Discuss **polymorphism** in general
- » Demo **examples** of polymorphic functions

# Polymorphism

# Recall: Polymorphism and Lists

'a list  $\rightarrow$  int

```
let rec length l =  
  match l with  
  | [] -> 0  
  | x :: xs -> 1 + length xs
```

*What is the type of the length function?*

Does this function depend on the values in the list?

# Recall: The List Type

[1;2;3]

**int list**

["1";"2";"3"]

**string list**

[[1;1];[2;2];[3;3]]

**int list list**

The list type is an example of a **parametrized** type

A function on lists is *polymorphic* (with respect to the list parameter) if it can be apply to a list parametrized by *any* type

For this, we need *type parameters* to stand for *any* type:

**'a, 'b, 'c, ...**

# Intuition: Int List Reversal

*Implement the function*

***reverse : int list -> int list***

*such that **reverse** **l** is the same as **l** but in reverse order*

# Implementation

```
let rec rev_int l =  
  match l with  
  | [] -> []  
  | x :: tl -> (rev_int tl) @ [x]  
  
let _ = assert (rev_int [1;2;3] = [3;2;1])
```

Take the first element of `l`

Reverse the `tl`

Append `[x]` to the end of reversed `tl`

# Intuition: String List Reversal

*Implement the function*

***reverse : string list -> string list***

*such that **reverse l** is the same as **l** but in reverse order*

# Implementation

```
let rec rev_string l =  
  match l with  
  | [] -> []  
  | x :: tl -> (rev_string tl) @ [x]  
  
let _ = assert (rev_string ["1";"2";"3"] = ["3";"2";"1"])
```

Take the first element of `l`

Reverse the `tl`

Append `[x]` to the end of reversed `tl`

# What's the Similarity?

```
let rec rev_int l =  
  match l with  
  | [] -> []  
  | x :: tl -> (rev_int tl) @ [x]
```

```
let rec rev_string l =  
  match l with  
  | [] -> []  
  | x :: tl -> (rev_string tl) @ [x]
```

Exact same implementation!

Copy/pasting code is *time consuming* and *error prone*

**Polymorphism** allows for better code reuse. The *same* function can be applied in multiple contexts

# Not all functions are polymorphic

```
let rec sum l =  
  match l with  
  | [] -> 0  
  | x :: xs -> x + sum xs
```

*Can this function be applied to a list parametrized by any type?*

**Answer:** No, it can only be applied to **int lists**

OCaml's type inference is good at "guessing" when functions are polymorphic

# Basic Example

*let id = fun x → x in*

```
let id = fun x -> x (fun x → x) (fun x → x)
let a = id 0
let b = id (0 = 0)
let c = id id
```

*id id*  
*(fun x → x)*

We want to be able to define functions that can be used in multiple contexts *and* that we can type check

Important: We can evaluate this if we *don't* type check

*But if we type-check, what should be the type of **id**?*

# Answer: Parametric Polymorphism

```
let id = fun x -> x
let a = id 0
let b = id (0 = 0)
let c = id id
```

Parametric polymorphism allows for functions which are agnostic to the types of its inputs (this is what OCaml does)

*For example, we can write a single identity function and use it with multiple types*

# Type Parameters

```
let id : 'a -> 'a = fun x -> x
```

The "parametric" part is the fact that types have *parameters*

**Type parameters** are instantiated at particular types according to the context

# Polymorphism and Type Inference

```
fun f -> fun x -> f (x + 1)
```

In OCaml, the type of an expression depends on how its subexpressions are *used*

OCaml can figure out what is the "most" polymorphic (general) type we can give to an expression

This is called **type inference**

There are many subtleties  
to this...

# One Subtlety: Dynamic Dispatch

```
let to_string (x : 'a) : string = ...  
(* This is not possible in OCaml *)
```

Parametric polymorphism cannot be used for *dispatch*

We can't write a polymorphic function that "checks the type" to see what to do

# Practice Problem

```
let rec f x = f (x + 1) in f
```

*What is the type of the above OCaml expressions?*

# Answer

```
let rec f x = f (x + 1) in f
```

# Practice Problem

```
let rec f x = f (f (x + 1)) in f
```

*What is the type of the above OCaml expressions?*

# Answer

```
let rec f x = f (f (x + 1)) in f
```

# Aside: Parametricity

*Can you write function of type '**a** -> 'a?*

*Can you write another?*

**Parametricity** refers to the (very cool) observation that polymorphic types often restrict the kinds of functions you can write in a *mathematically rigorous sense*

# Challenge: Parametricity in Lists

*Can you write function of type:*

***'a list -> 'a list?***

*How many such functions can you write?*

*What can these functions do?*

# Summary

Polymorphism allows functions to be agnostic to (parts of) the types of its parameters

Parametric polymorphism does not allow for dynamic dispatch, it must given the *same* implementation at any particular type